Lecture 26.04: Linear Regression GD (Step 2 — MSE Cost Calculation)

📌 Topic Overview

In this lecture, Shraddha ma'am covers **Step 2 of Gradient Descent**: formulating the **Mean Squared Error (MSE)** loss function that quantifies the difference between predictions and actual targets.

📐 Mathematical Formulation

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Where:

- m = Number of training samples.
- $\hat{y}^{(i)} = X^{(i)}w + b$ = Predicted value for sample i.
- $y^{(i)}$ = Ground truth label for sample i.
- The $\frac{1}{2}$ **factor** cancels the exponent 2 when taking derivatives during calculus: $\frac{d}{dz}(\frac{1}{2}z^2) = z$.

💻 Python Implementation

```
import numpy as np

def compute_mse_cost(y_true, y_pred):
    """
    Step 2 of Gradient Descent: Compute Mean Squared Error (MSE) Cost.
    Formula: J = (1 / (2 * m)) * sum((y_pred - y_true)^2)

    The 1/2 factor simplifies calculus derivatives.
    """
    m = len(y_true)
    if m == 0:
        return 0.0
```

```
    error_residuals = y_pred - y_true
    cost = (1.0 / (2.0 * m)) * np.sum(error_residuals ** 2)
    return float(cost)

# --- Function Call & Testing ---
actual_vals    = np.array([3.0, -0.5, 2.0, 7.0])
predicted_vals = np.array([2.5,  0.0, 2.0, 8.0])

loss = compute_mse_cost(actual_vals, predicted_vals)
print(f"Calculated MSE Loss (J): {loss:.4f}")
```

🔑 Key Takeaways

1. Squaring errors ensures that positive and negative residuals do not cancel each other out.
2. Squaring punishes large outlier mistakes more severely than small ones.